Algorithm 1 Training the Diffusion Prior for unCLIP

Require: Pre-trained CLIP model, dataset of text-image pairs $\{(y, x)\}$, PCA matrix P for dimensionality reduction, Transformer model T_θ, diffusion timesteps T, noise schedule $\beta_1, \ldots, \beta_T$

Ensure: Trained prior model parameters θ

Initialize θ randomly
for each training epoch **do**
  for each batch of (y, x) **do**
    Encode text y with CLIP text encoder: $z_t \leftarrow \text{CLIP}_{\text{text}}(y)$
    Encode image x with CLIP image encoder: $z_i \leftarrow \text{CLIP}_{\text{image}}(x)$
    Reduce dimensionality: $z_i \leftarrow P \cdot z_i$ {PCA to 319 components}
    Sample timestep $t \sim \text{Uniform}(1, T)$
    Sample noise $\epsilon \sim \mathcal{N}(0, I)$
    Compute noised embedding: $z_{i,t} \leftarrow \sqrt{\bar{\alpha}_t} z_i + \sqrt{1 - \bar{\alpha}_t}\epsilon$ {Using noise schedule}
    Prepare input sequence: $s \leftarrow [y, z_t, t, z_{i,t}]$
    Predict unnoised embedding: $\hat{z}_i \leftarrow T_\theta(s)$
    Compute loss: $L \leftarrow \|\hat{z}_i - z_i\|_2^2$
    Update θ using gradient descent on L
  end for
end for
return θ
=0

Algorithm 2 Training the Diffusion Decoder for unCLIP

Require: Pre-trained CLIP model, dataset of text-image pairs $\{(y, x)\}$, PCA matrix P, diffusion model D_ϕ (modified GLIDE), diffusion timesteps T, noise schedule $\beta_1, \ldots, \beta_T$

Ensure: Trained decoder model parameters ϕ

0: Initialize ϕ randomly
0: **for** each training epoch **do**
0: **for** each batch of (y, x) **do**
0: Encode text y with CLIP text encoder: $z_t \leftarrow \text{CLIP}_{\text{text}}(y)$
0: Encode image x with CLIP image encoder: $z_i \leftarrow \text{CLIP}_{\text{image}}(x)$
0: Reduce dimensionality: $z_i \leftarrow P \cdot z_i$
0: Sample $p \sim \text{Uniform}(0, 1)$
0: **if** $p < 0.1$ **then**
0: Set $z_i \leftarrow 0$ {Classifier-free guidance}
0: **end if**
0: **if** $p < 0.5$ **then**
0: Set $y \leftarrow \emptyset$ {Drop text caption}
0: **end if**
0: Project z_i to 4 tokens: $c \leftarrow \text{Project}(z_i)$
0: Encode text y: $y_{\text{enc}} \leftarrow \text{GLIDE}_{\text{text}}(y)$
0: Concatenate: $s \leftarrow [y_{\text{enc}}, c]$
0: Sample timestep $t \sim \text{Uniform}(1, T)$
0: Sample noise $\epsilon \sim \mathcal{N}(0, I)$
0: Compute noised image: $x_t \leftarrow \sqrt{\bar{\alpha}_t} x + \sqrt{1 - \bar{\alpha}_t}\epsilon$
0: Add z_i to timestep embedding: $t_{\text{emb}} \leftarrow t_{\text{emb}} + \text{Project}(z_i)$
0: Predict noise: $\hat{\epsilon} \leftarrow D_\phi(x_t, t_{\text{emb}}, s)$
0: Compute loss: $L \leftarrow \|\hat{\epsilon} - \epsilon\|_2^2$
0: Update ϕ using gradient descent on L
0: **end for**
0: **end for**
0: **return** ϕ =0

Algorithm 3 Training Upsampling Diffusion Models for unCLIP

Require: Dataset of images at resolutions r_{low} (e.g., 64×64) and r_{high} (e.g., 256×256 or 1024×1024), diffusion model U_ψ, diffusion timesteps T, noise schedule $\beta_1, \ldots, \beta_T$

Ensure: Trained upsampler model parameters ψ

0: Initialize ψ randomly
0: **for** each training epoch **do**
0: **for** each batch of $(x_{\text{low}}, x_{\text{high}})$ **do**
0: Sample timestep $t \sim \text{Uniform}(1, T)$
0: Sample noise $\epsilon \sim \mathcal{N}(0, I)$
0: Compute noised image: $x_{\text{high},t} \leftarrow \sqrt{\bar{\alpha}_t} x_{\text{high}} + \sqrt{1 - \bar{\alpha}_t}\epsilon$
0: Predict noise: $\hat{\epsilon} \leftarrow U_\psi(x_{\text{high},t}, t, x_{\text{low}})$
0: Compute loss: $L \leftarrow \|\hat{\epsilon} - \epsilon\|_2^2$
0: Update ψ using gradient descent on L
0: **end for**
0: **end for**
0: **return** ψ =0

Algorithm 4 Inference Pipeline for unCLIP

Require: Pre-trained CLIP model, trained prior T_θ, trained decoder D_ϕ, upsamplers U_{ψ_1} (64→256), U_{ψ_2} (256→1024), text prompt y, prior guidance scale $g_p = 4.0$, decoder guidance scale $g_d = 8.0$, timesteps T_p, T_d, T_{u1}, T_{u2}

Ensure: Generated image x_{final}

Encode text: $z_t \leftarrow \text{CLIP}_{\text{text}}(y)$
Initialize noise: $z_{i,T_p} \sim \mathcal{N}(0, I)$
for $t = T_p$ to 1 **do**
    Prepare sequence: $s \leftarrow [y, z_t, t, z_{i,t}]$
    Predict unnoised embedding: $\hat{z}_i \leftarrow T_\theta(s)$
    Compute guided prediction: $\hat{z}_i \leftarrow (1 + g_p) \cdot \hat{z}_i - g_p \cdot T_\theta([y, 0, t, z_{i,t}])$
    Update $z_{i,t-1}$ using diffusion step (e.g., DDIM)
end for
Obtain $z_i \leftarrow z_{i,0}$
Initialize noise: $x_{T_d} \sim \mathcal{N}(0, I)$ {64×64 resolution}
Project z_i to 4 tokens: $c \leftarrow \text{Project}(z_i)$
Encode text y: $y_{\text{enc}} \leftarrow \text{GLIDE}_{\text{text}}(y)$
Concatenate: $s \leftarrow [y_{\text{enc}}, c]$
for $t = T_d$ to 1 **do**
    Add z_i to timestep embedding: $t_{\text{emb}} \leftarrow t_{\text{emb}} + \text{Project}(z_i)$
    Predict noise: $\hat{\epsilon} \leftarrow D_\phi(x_t, t_{\text{emb}}, s)$
    Compute guided prediction: $\hat{\epsilon} \leftarrow (1 + g_d) \cdot \hat{\epsilon} - g_d \cdot D_\phi(x_t, t_{\text{emb}}, [0, 0])$
    Update x_{t-1} using diffusion step (e.g., DDIM)
end for
Obtain $x_{\text{low}} \leftarrow x_0$ {64×64 image}
Initialize noise: $x_{T_{u1}} \sim \mathcal{N}(0, I)$ {256×256 resolution}
for $t = T_{u1}$ to 1 **do**
    Predict noise: $\hat{\epsilon} \leftarrow U_{\psi_1}(x_t, t, x_{\text{low}})$
    Update x_{t-1} using diffusion step
end for
Obtain $x_{\text{mid}} \leftarrow x_0$ {256×256 image}
Initialize noise: $x_{T_{u2}} \sim \mathcal{N}(0, I)$ {1024×1024 resolution}
for $t = T_{u2}$ to 1 **do**
    Predict noise: $\hat{\epsilon} \leftarrow U_{\psi_2}(x_t, t, x_{\text{mid}})$
    Update x_{t-1} using diffusion step
end for
Obtain $x_{\text{final}} \leftarrow x_0$ {1024×1024 image}
return x_{final} =0